FROM-SCRATCH LEARNING MANUAL

POWER BI · END TO END

SaaS Sales Performance Dashboard

A complete, step-by-step guide to building the dashboard from zero —
planning, Power Query, data modeling, DAX, design, and business insight.

Project: SaaS Sales Performance Analytics Dashboard

Dataset: Amazon AWS SaaS Sales (Kaggle)

Prepared for: Akshay Juluru

Level: Beginner → Job-ready

Table of Contents

HOW TO USE THIS GUIDE

Work top to bottom the first time — each chapter builds on the last. The build order is deliberate: **plan → import → transform → model → measure → visualize**. Skipping ahead to visuals before the model exists is the number-one beginner mistake, and this guide is structured to stop you doing it. Keep Power BI Desktop open beside this document and do each step as you read it.

1 Dashboard Planning

> *A dashboard is a decision tool, not a chart gallery. Before touching Power BI, decide what decision it drives and for whom. Ten minutes of planning saves hours of rebuilding.*

1.1 Business Objectives

This dashboard exists to give the leadership of a B2B SaaS company (it sells sales & marketing software to other businesses) a single, trustworthy view of sales performance. The company is profitable overall but cannot see *where* profit comes from or *where* margin leaks. Every visual we build serves one of these five objectives:

- Identify the most profitable regions, industries, segments, and products.
- Quantify how discounting affects margin (the core hidden problem).
- Surface loss-making products so they can be fixed or dropped.
- Measure revenue concentration — how dependent we are on a few customers.
- Track performance over time (growth and seasonality).

BEST PRACTICE

If a visual you are about to build does not answer one of the objectives above, do not build it. This single rule keeps dashboards focused and is exactly what separates a professional dashboard from a cluttered one.

1.2 KPIs

KPIs are the headline numbers a leader reads in three seconds, plus the diagnostic numbers they drill into. We split them deliberately.

Tier	KPI	What it tells the business
Headline	Total Sales	Top-line revenue
Headline	Total Profit	What we actually keep
Headline	Profit Margin %	Efficiency of the revenue
Headline	Total Orders (distinct)	Deal volume
Headline	Average Order Value	Deal size
Diagnostic	Average Discount %	Discount discipline
Diagnostic	Sales/Profit by Region, Industry, Segment, Product	Where performance concentrates
Diagnostic	Discount vs Margin	The margin-leak story
Diagnostic	Revenue concentration (top-N share)	Customer-dependency risk
Diagnostic	YoY / MoM growth	Trajectory

1.3 Target Users

User	What they open the dashboard for
VP / Director of Sales (primary)	Overall health; which segments to prioritize
Regional Sales Managers	Their region's numbers versus others
Finance / RevOps	Margin discipline and discount governance
Marketing	Which industries and segments actually convert
C-suite	A one-glance summary of top-line health

WHY THIS MATTERS

Different users need different depth. The primary user (VP Sales) drives the layout of Page 1. Secondary users get dedicated pages or slicers. Designing "for everyone" produces a dashboard that works for no one.

1.4 Wireframe & Page Planning

We use three pages plus a hidden drill-through page. Sketch the layout before building — here is the plan for Page 1 (the Executive Overview), the most important screen.

Wireframe — Page 1 (Executive Overview). Header + navigation, a row of 5 KPI cards, a slicer strip, then four charts in two bands.

Page plan (the full report)

- **Page 1 — Executive Overview:** KPI cards, trend, region, profit-by-product, top customers.

- **Page 2 — Product & Discount:** the discount-vs-margin story, product profitability, product×region matrix.
- **Page 3 — Customer Analysis:** top customers, revenue concentration (Pareto), industry×segment matrix.
- **Hidden — Customer Detail:** a drill-through target showing one customer's full picture.

1.5 Best Layout Principles

- **Read top-left first.** Eyes land there — put your KPI cards and title at the top. Detail flows down and right.
- **Summary above detail.** Big numbers up top, breakdowns below, row-level detail on drill-through.
- **Group related visuals.** Keep the trend and region charts together; keep product visuals together.
- **Leave breathing room.** White space is not wasted space — it makes the dashboard scannable.
- **One story per page.** Page 1 = health, Page 2 = discounting, Page 3 = customers. Don't mix.

2 Importing Data

> *Getting data in cleanly is the foundation. A wrong data-type or locale setting here silently breaks every number downstream, so slow down for this step.*

2.1 How to Import the Dataset

We import the cleaned CSV produced by the project's Python pipeline (`saas_sales_clean.csv`). That file is already de-duplicated, typed, and feature-engineered, so Power BI's job is modeling and visuals, not cleaning.

1. Open Power BI Desktop and close the splash screen.
2. On the **Home** ribbon click **Get data**.
3. Choose **Text/CSV** and browse to `saas_sales_clean.csv`.
4. A preview window appears showing the first rows and the detected types.
5. Click **Transform Data** (not Load) so you land in Power Query and can verify everything first.

VISUAL PLACEHOLDER — GET DATA DIALOG

The "Get Data" window with categories on the left (File, Database, Power Platform, Azure, Online Services). "Text/CSV" is the first option under File. The CSV preview shows columns like order_id, order_date, sales, profit, discount with a small type icon on each header.

2.2 File Formats You'll Meet

Source	When to use it
Text/CSV	A single flat export (our case). Simplest, most portable.
Excel	Multi-sheet workbooks; when data lives in .xlsx.
Database (PostgreSQL, SQL Server…)	Production. Connect straight to the star schema from the SQL layer.
Folder	Many files with the same shape (e.g., one CSV per month) — combine them automatically.
Web / API	Live external data.

BEST PRACTICE

For your portfolio, loading the flat CSV is fine and reliable. For the "production" version of the story, use **Get data → PostgreSQL** and import the `fact_sales` + dimension tables directly — it demonstrates the full pipeline (Python → SQL → Power BI) and impresses interviewers.

2.3 Data Source Settings

In the CSV preview, three settings matter:

- **Delimiter** — Comma for our file. Power BI usually detects it.

- **Data Type Detection** — "Based on first 200 rows" is the default. It can guess a type wrong from a small sample, so we verify manually in Power Query.
- **File Origin / Encoding** — Use **65001: Unicode (UTF-8)**. Wrong encoding turns accented names into garbled characters.

> **COMMON MISTAKE — LOCALE**
> Power BI parses dates and numbers using your machine's *regional* locale. If your locale expects DD/MM/YYYY but the file is MM/DD/YYYY (or vice-versa), dates silently import wrong or as errors. Fix it under **File → Options → Regional settings**, or set the type "using locale" in Power Query (right-click column → Change Type → Using Locale).

2.4 Common Mistakes & Best Practices

Mistake	Do this instead
Loading the raw Kaggle file and cleaning inside Power BI	Load the pipeline's cleaned CSV; keep cleaning in the Python layer (one source of truth)
Clicking "Load" immediately	Click "Transform Data" and confirm types first
Numbers imported as text	Check the header type icon — a number stored as text (`ABC`) will not sum in visuals
Importing columns you'll never use	Remove high-cardinality noise (e.g., `license`) to keep the model light

3 Power Query — Every Transformation

> *Power Query is where raw data becomes analysis-ready. It records each action as an "Applied Step" (in the M language) and replays them every refresh, so cleaning is automatic and repeatable. Below is every transformation you need, what it does, and why.*

THE APPLIED STEPS PANEL

Every click you make appears on the right under **Applied Steps**. You can rename, reorder, or delete steps. This is your undo history and your documentation in one. Give steps clear names.

3.1 Remove Duplicates

What: deletes rows that are exact copies. **Why:** duplicate transactions double-count revenue. **How:** select the column(s) that define uniqueness → Home → Remove Rows → **Remove Duplicates**. To dedupe whole rows, select all columns first.

CAREFUL

"Remove Duplicates" keeps the *first* occurrence and is order-sensitive. Sort first if which row you keep matters. Our cleaned dataset has no duplicates (the Python layer verified this) — but you must know the tool.

3.2 Handling Null Values

What: deal with empty cells. **Why:** nulls break aggregations and joins. Options:

- **Remove:** filter out rows where a key column is null (Home → Remove Rows → Remove Blank Rows, or filter the column).
- **Replace:** right-click column → Replace Values (e.g., null → 0 for a numeric measure where blank truly means zero).
- **Fill Down/Up:** Transform → Fill → Down, to carry a value into blank cells below it (useful for merged-cell exports).

JUDGMENT

Never blindly replace nulls with 0. A null *discount* may mean "no discount" (→ 0 is right), but a null *profit* means "unknown" (→ 0 is wrong and hides a problem). Decide per column.

3.3 Changing Data Types

What: tell Power BI whether a column is text, number, date, etc. **Why:** this is the single most important Power Query step — wrong types break math, dates, and relationships. **How:** click the type icon on the header (or right-click → Change Type). Set:

- `sales, profit, discount, profit_margin` → **Decimal Number**
- `quantity, year, month, quarter, date_key` → **Whole Number**
- `order_date` → **Date**
- everything else → **Text**

NUMBER-AS-TEXT TRAP

If `sales` imports as text, your "Total Sales" card will error or show blank. Always confirm measures are a number type here, before modeling.

3.4 Splitting & Merging Columns

Split — Home → Split Column (by delimiter, by number of characters, by position). Example: split a full "City, Country" field into two. **Merge** — select two columns → Add Column → Merge Columns, choosing a separator. Example: combine `region` + `subregion` into a single "Region – Subregion" label for a cleaner slicer.

3.5 Replacing Values

What: swap one value for another across a column. **Why:** standardize inconsistent categories. **How:** right-click column → Replace Values. Example: unify "USA", "U.S.", and "United States" into one value so they don't split your region totals into three.

3.6 Creating Custom Columns

What: add a new column from a formula (M language). **Why:** derive fields the source doesn't have. **How:** Add Column → Custom Column. Example — a discount band computed at load time:

```
if [discount] = 0 then "0%"
else if [discount] <= 0.10 then "1-10%"
else if [discount] <= 0.20 then "11-20%"
else if [discount] <= 0.30 then "21-30%"
else "30%+"
```

POWER QUERY VS DAX

Static attributes that never change (like a discount band) can live as a Power Query column — it's computed once at load and keeps your model tidy. Numbers that must react to slicers (like Total Sales) must be DAX *measures*, not columns. Rule of thumb: descriptive text → Power Query; reactive calculations → DAX.

3.7 Conditional Columns

The same idea as a custom column but built through a friendly UI with no code: Add Column → **Conditional Column**, then add "if / else if" rules. Example: create an `is_loss` flag — if `profit < 0` then "Loss" else "Profit". Great for beginners who aren't comfortable writing M yet.

3.8 Pivot & Unpivot

Unpivot turns wide columns into tall rows — the shape Power BI prefers. If your data had one column per month (Jan, Feb, Mar…), select those columns → Transform → **Unpivot Columns** to get tidy "Month / Value" rows. **Pivot** is the reverse (rows → columns), used sparingly, e.g., to build a small summary table.

WHY UNPIVOT MATTERS

Power BI and DAX are built for "long" (tidy) data — one row per observation. Wide, spreadsheet-style data with a column per period is hard to chart. Unpivoting is the fix, and it's a common interview topic.

3.9 Append Queries

What: stack tables with the same columns on top of each other (adds rows). **Why:** combine files that share a structure — e.g., `sales_2022` + `sales_2023` into one table. **How:** Home → **Append Queries** → choose the tables. Think of it as UNION in SQL.

3.10 Merge Queries

What: join two tables side by side on a matching key (adds columns). **Why:** bring in lookup attributes — e.g., attach a "region manager" from a separate table using `region` as the key. **How:** Home → **Merge Queries** → pick the key column in each table and a join kind (Left Outer is the usual default). Think of it as JOIN in SQL.

MODELING TIP

Prefer building proper table *relationships* in the model over merging everything into one giant table. Merge only when you genuinely need extra columns on a single table; otherwise keep dimensions separate (see Chapter 4).

3.11 Group By

What: aggregate rows to a coarser grain. **Why:** build summaries — e.g., roll order lines up to one row per order. **How:** Transform → **Group By**, choose the grouping column(s) and the aggregation (Sum, Count, etc.).

THE ORDER-COUNT TRAP

Our data has multiple rows per order. To count orders correctly, group by `order_id` or, in DAX, use `DISTINCTCOUNT(order_id)` — never a plain row count. Counting rows overstates the number of orders and is a classic interview "gotcha".

3.12 Date, Text & Numeric Transformations

Date transformations

Select `order_date` → Add Column → **Date** → extract Year, Month, Quarter, Day, Day Name, etc. (We build a dedicated Date table in DAX instead — Chapter 4 — but these extractions are handy for quick fields.)

Text transformations

Transform → Format → **Trim** (remove leading/trailing spaces), **Clean** (strip non-printing characters), and case functions (UPPER/lower/Capitalize). Trimming text keys prevents "Finance " and "Finance" from being treated as two categories.

Numeric transformations

Transform → **Rounding**, **Standard** (add/subtract/multiply/divide), and **Statistics**. Useful for rounding a rate column or scaling a value.

3.13 Finish — Close & Apply

When every column is correctly typed and any derived columns exist, click **Close & Apply** (Home, top-left). Power Query runs all Applied Steps once and loads the finished table into the model. You only reopen Power Query when the data shape needs to change again.

> **INTERVIEW ANGLE**
> Be ready to explain the difference between Power Query (M, transforms data as it loads, runs once per refresh) and DAX (calculates live as users interact). Mixing them up is the most common conceptual gap in junior candidates.

4 Data Modeling

> *The model is the engine under the dashboard. Get the tables and relationships right and DAX becomes easy; get them wrong and every measure fights you. This is the highest-leverage chapter for interviews.*

4.1 Relationships

A relationship connects two tables on a shared column so a filter on one table flows to the other. When you slice by a Year in the `Date` table, the relationship carries that filter to `Sales` and every measure recalculates. Without relationships, your tables are islands and slicers do nothing.

4.2 Cardinality

Cardinality describes how rows on one side match rows on the other:

Type	Meaning	Example
Many-to-one (*:1)	Many fact rows point to one dimension row	Many sales → one date; the normal, healthy case
One-to-one (1:1)	Each row matches exactly one row	Rare; usually a sign two tables should be merged
Many-to-many (*:*)	Both sides have duplicates	Use sparingly — can produce ambiguous or inflated results

RULE OF THUMB

Almost every relationship in a well-built model is **many-to-one** from the fact table to a dimension. If you see many-to-many, pause and ask whether your model is really a star.

4.3 Cross-Filter Direction

Direction controls which way filters travel across a relationship.

Single direction: the dimension filters the fact (recommended)

Cross-filter direction — Single (the default and the right choice for star schemas).

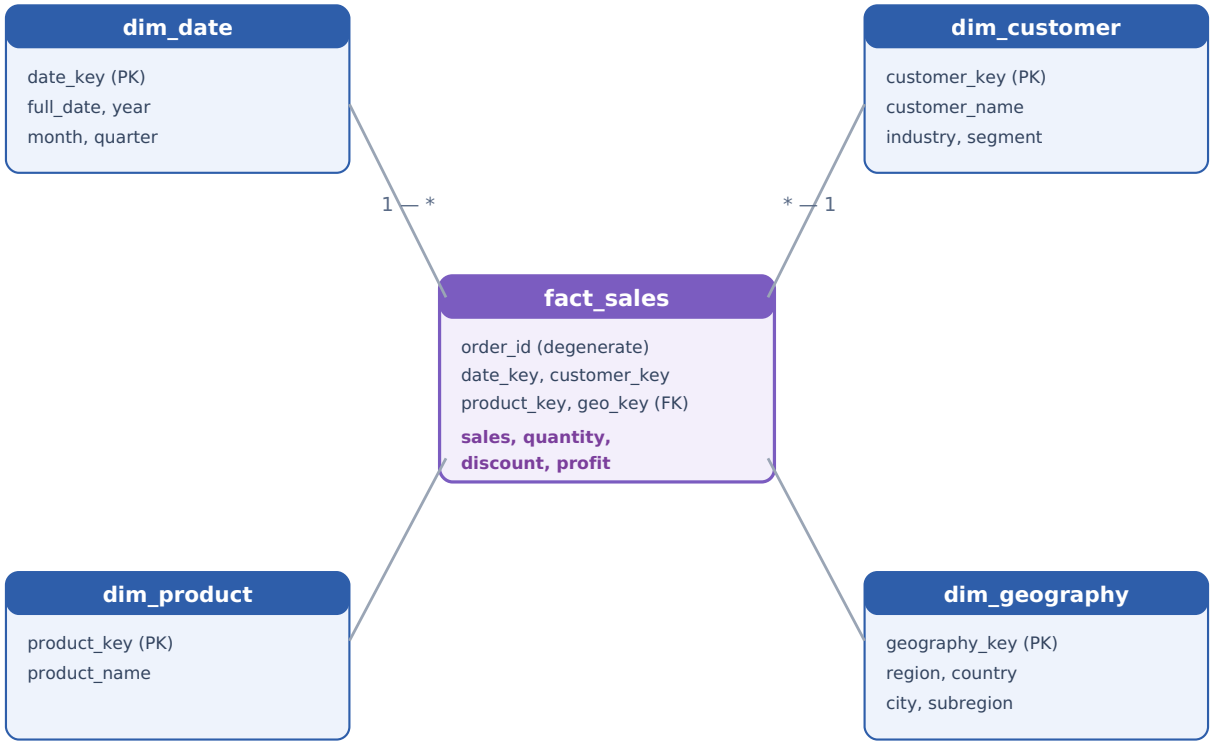
- **Single (recommended):** the dimension filters the fact. Predictable and fast.
- **Both (bidirectional):** filters travel both ways. Occasionally needed, but it can create ambiguity, circular paths, and slow queries. Use only with a clear reason.

4.4 Fact vs Dimension Tables

- **Fact table** — the events you measure. Long and narrow: mostly keys plus numeric measures. Ours is `Sales` (one row per order line: sales, quantity, discount, profit).
- **Dimension tables** — the descriptive "by what?" context you slice by: `Date`, `Customer`, `Product`, `Geography`. Short and wide: descriptive attributes.

4.5 Star Schema

A star schema is one central fact table surrounded by dimension tables, each joined directly to the fact. It looks like a star. It is the shape Power BI is optimized for — fast, simple to filter, and easy to reason about.



Our star schema — `fact_sales` at the center, four dimensions around it. `order_id` is a degenerate dimension kept in the fact so order counts stay correct.

DEGENERATE DIMENSION

`order_id` is an ID we keep inside the fact table without a dimension table of its own. It lets us compute `DISTINCTCOUNT(order_id)` for true order counts even though one order spans several fact rows.

4.6 Snowflake Schema (and why we avoid it here)

A snowflake schema splits dimensions further into sub-dimensions (e.g., `Product → Category → Department` as separate linked tables). It normalizes data and reduces redundancy, but it adds joins, which makes queries slower and DAX more complex. For dashboards, a **star** is almost

always preferred; snowflake is justified only for very large or highly normalized enterprise models.

	Star	Snowflake
Structure	Dimensions join directly to the fact	Dimensions split into sub-tables
Speed	Faster (fewer joins)	Slower (more joins)
DAX simplicity	Simpler	More complex
Use when	Most BI dashboards (our case)	Very large, highly normalized models

4.7 Building Our Model (step by step)

If you loaded the single flat `Sales` table, you still need a Date table for time intelligence. Build it and connect it:

1. Modeling tab → **New table** → paste the `Date =` DAX (see Chapter 5.4) and press Enter.
2. Go to **Model view** (linked-boxes icon on the left).
3. Drag `Sales[order_date]` onto `Date[Date]` to create the relationship. Confirm it reads many-to-one, single direction.
4. Select the `Date` table → Table tools → **Mark as date table** → date column = `Date`.
5. Hide key columns you never display (right-click → Hide) to keep the field list clean.

4.8 Best Practices

- Model as a **star**; keep the fact narrow (keys + measures) and put descriptive text in dimensions.
- Use **single-direction** relationships unless you have a specific reason not to.
- Always build and **mark a Date table** for time intelligence.
- Hide surrogate keys and technical columns from the report view.
- Give tables and columns clean, human names — they show up in visuals.

4.9 Common Mistakes

Mistake	Consequence
One giant flat table	Loses slicing power, bloats the model, no reusable dimensions
Bidirectional filters everywhere	Ambiguous filter paths, wrong numbers, slow reports
Forgetting to mark the Date table	Time-intelligence functions (YoY, YTD) misbehave or error
Relating on a text name instead of an ID	Fragile joins; typos split your data

5 DAX — Every Measure Explained

> *DAX is the calculation language of Power BI. This chapter gives every measure the dashboard needs, and for each: why it exists, the formula, what each function does, and how it shows up on screen.*

5.1 The Two DAX Concepts You Must Know

- **Measure vs calculated column.** A *column* is computed row-by-row and stored (use for static attributes). A *measure* is computed on the fly for whatever is on screen (use for all your KPIs). Measures are the heart of DAX.
- **Filter context.** A measure returns a different number depending on the filters around it (the slicers, the row of a table, the axis of a chart). "Total Sales" on a region bar shows that region's sales because the bar sets a filter context of one region. Understanding this explains almost everything in DAX.

ORGANIZE YOUR MEASURES

Create an empty table (New table → `_Measures = {BLANK()}`), hide its column, and store every measure there so they're easy to find. Add each measure with **New measure**.

5.2 Basic Measures

Total Sales

```
Total Sales = SUM(Sales[sales])
```

Why: the headline revenue number. **Function — SUM:** adds a column across the current filter context. **On the dashboard:** the first KPI card; the value behind every sales chart.

Total Profit

```
Total Profit = SUM(Sales[profit])
```

Why: what the business keeps after costs. **On the dashboard:** second KPI card; the profit-by-product bar.

Total Orders (the important one)

```
Total Orders = DISTINCTCOUNT(Sales[order_id])
```

Why: deal volume. **Function — DISTINCTCOUNT:** counts unique values, ignoring repeats. **Critical:** one order spans multiple rows, so a plain `COUNT` would overstate orders. **On the dashboard:** the "Orders" KPI and the denominator of Average Order Value.

INTERVIEW ANGLE

Expect "how do you count orders when one order has many lines?" The answer is `DISTINCTCOUNT` on the order ID. Saying "COUNT of rows" fails the question.

Total Quantity & Total Customers

```
Total Quantity  = SUM(Sales[quantity])
Total Customers = DISTINCTCOUNT(Sales[customer_id])
```

Why: units sold, and how many distinct customers we serve (again DISTINCTCOUNT, for the same reason as orders).

5.3 Intermediate Measures

Profit Margin %

```
Profit Margin % = DIVIDE([Total Profit], [Total Sales])
```

Why: efficiency — profit per dollar of sales. **Function — DIVIDE:** divides safely and returns blank (not an error) if the denominator is zero. **Critical:** compute margin as total profit ÷ total sales, *never* as the average of row-level margins — averaging ratios gives a wrong answer. Format the measure as a percentage. **On the dashboard:** the Margin KPI and the discount-vs-margin chart.

> **MEASURE, NOT COLUMN**
> If you drag a pre-computed row-level margin column into a card, Power BI averages it and lies to you. Use this measure instead.

Average Order Value

```
Avg Order Value = DIVIDE([Total Sales], [Total Orders])
```

Why: typical deal size. Note it reuses `[Total Orders]`, so it inherits the correct distinct-count logic.

Average Discount %

```
Avg Discount % = AVERAGE(Sales[discount])
```

Why: discount discipline. `discount` is stored 0–1, so format this measure as a percentage. **Function — AVERAGE:** the mean of a column in context.

5.4 Time-Intelligence Measures

These need the marked Date table from Chapter 4. First, the Date table itself:

```
Date =
VAR BaseCalendar = CALENDARAUTO()
RETURN
ADDCOLUMNS(
    BaseCalendar,
    "Year", YEAR([Date]),
    "Month", MONTH([Date]),
    "Month Name", FORMAT([Date], "MMM"),
    "Quarter", "Q" & FORMAT([Date], "Q")
)
```

CALENDARAUTO scans your data and builds one row per day; **ADDCOLUMNS** adds Year/Month/etc.; **FORMAT** turns a date into a text label.

Sales Last Year

```
Sales LY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
```

Why: the comparison baseline. **CALCULATE** changes the filter context of a measure; **SAMEPERIODLASTYEAR** shifts the date filter back exactly one year. **On the dashboard:** feeds YoY growth.

Sales YoY %

```
Sales YoY % = DIVIDE([Total Sales] - [Sales LY], [Sales LY])
```

Why: are we growing? Shows the percentage change versus the same period last year — a headline trend indicator.

Sales YTD

```
Sales YTD = TOTALYTD([Total Sales], 'Date'[Date])
```

Why: cumulative performance this year. **TOTALYTD** sums from Jan 1 to the current point in context.

Sales MoM %

```
Sales MoM % =
VAR PrevMonth = CALCULATE([Total Sales], DATEADD('Date'[Date], -1, MONTH))
RETURN DIVIDE([Total Sales] - PrevMonth, PrevMonth)
```

Why: short-term momentum. **DATEADD** shifts the date context by a chosen interval (here −1 month); **VAR/RETURN** stores an intermediate value for readable formulas.

5.5 Ranking

Product Rank

```
Product Rank = RANKX(ALL(Sales[product]), [Total Sales], , DESC)
```

Why: order products best-to-worst. **RANKX** ranks each item by an expression; **ALL** removes the row's own filter so the ranking is across all products, not just the current one. **On the dashboard:** label top performers or filter to a Top-N.

5.6 Running Totals

Running Total Sales

```
Running Total Sales =
CALCULATE(
    [Total Sales],
    FILTER(ALLSELECTED('Date'[Date]), 'Date'[Date] <= MAX('Date'[Date]))
)
```

Why: a cumulative "how are we tracking?" line. **FILTER** builds a custom set of dates (every date up to the current one); **ALLSELECTED** respects slicers but ignores the visual's own row; **MAX** finds the latest date in the current context.

5.7 Advanced Measures

% of Total Sales

```
% of Total Sales = DIVIDE([Total Sales], CALCULATE([Total Sales], ALLSELECTED(Sales)))
```

Why: each item's share of the whole. **ALLSELECTED** gives the visible grand total as the denominator.

Loss-Making Sales

```
Loss-Making Sales = CALCULATE([Total Sales], Sales[profit] < 0)
```

Why: how much revenue comes from unprofitable deals. The `profit < 0` argument becomes a filter inside CALCULATE. Computing it from profit directly avoids depending on any flag column.

Cumulative % of Sales (for the Pareto chart)

```
Cumulative % of Sales =
VAR CurrentCustomerSales = [Total Sales]
VAR AllCustomers =
    ADDCOLUMNS(ALLSELECTED(Sales[customer]), "@sales", [Total Sales])
VAR RankedAtOrAbove =
    FILTER(AllCustomers, [@sales] >= CurrentCustomerSales)
RETURN
DIVIDE(
    SUMX(RankedAtOrAbove, [@sales]),
    CALCULATE([Total Sales], ALLSELECTED(Sales))
)
```

Why: the classic "do a few customers drive most revenue?" curve. **SUMX** iterates a table and sums an expression; combined with the filter, it accumulates the revenue of all customers ranked at or above the current one. **On the dashboard:** the line on the Pareto combo chart in Chapter 6.

5.8 Dynamic Titles

```
Dynamic Title =
"Sales Performance — " &
IF(ISFILTERED('Date'[Year]), SELECTEDVALUE('Date'[Year]), "All Years")
```

Why: a title that reacts to slicers ("Sales Performance — 2023"). **SELECTEDVALUE** returns the single selected value (or a fallback); **ISFILTERED** tests whether a filter is applied. Set a visual's title to "conditional formatting → based on this measure".

5.9 KPI Calculations

KPI cards simply display a measure — but you can make them react. A status measure can drive the card's color:

```
Margin Status =
VAR m = [Profit Margin %]
RETURN IF(m < 0.10, "Below Target", "On Target")
```

Use it in conditional formatting on the card's font color (red below target, green on target) so the KPI signals health at a glance.

> **DAX BEST PRACTICES**
>
> Reuse base measures (build `Avg Order Value` from `[Total Sales]` and `[Total Orders]`) rather than rewriting logic; always use `DIVIDE` instead of `/`; use `VAR` for readability; and name measures in plain English.

6 Dashboard Development (Page by Page)

> *Now the visible part. For every visual we cover which type to use, why, where to place it, and how to build it. The golden rule from Chapter 1 still applies: every visual answers a real question.*

6.1 The Universal Visual Workflow

1. On the report canvas, click a visual icon in the **Visualizations** pane (or Insert → Visual).
2. Drag fields from the **Data** pane into the visual's wells (Axis, Values, Legend, etc.).
3. Resize and position it (drag corners; use arrow keys for fine nudges).
4. Open the **Format** pane (paint-roller icon) to set title, colors, labels, and borders.
5. Right-click → Sort by to control ordering.

> **ALIGNMENT SHORTCUT**
> Select multiple visuals (Ctrl+click) → Format → Align to snap them to a shared edge. This alone makes a dashboard look professional.

6.2 Page 1 — Executive Overview

Header & navigation (top band)

Insert → Text box for the title "SaaS Sales Performance". Add a colored rectangle (Insert → Shapes) behind it as a header bar. Place page-navigation buttons on the right (built in 6.8).

KPI cards — the five headline numbers

Visual: Card. **Why:** a single big number is the fastest thing to read. **Build:** Card visual → drag one measure into Fields. Repeat five times for Total Sales, Total Profit, Profit Margin %, Total Orders, Avg Order Value. **Placement:** a row of five equal cards directly under the header. **Format:** large bold value, small grey category label, subtle border (the theme handles most of this).

Sales & Profit Trend

Visual: Line chart. **Why:** lines show change over time better than any other visual. **Build:** Axis = `Date[Month Year]`; Values = `Total Sales` and `Total Profit`. **Placement:** left of the middle band, wider than its neighbor (roughly 60% width). **Sorting:** ensure the axis sorts chronologically (Month Year sorted by a year-month key — Chapter 3.12).

Sales & Profit by Region

Visual: Clustered bar chart. **Why:** bars compare categories cleanly. **Build:** Axis = `region`; Values = `Total Sales`, `Total Profit`. **Placement:** right of the trend chart. **Sort:** descending by sales.

Profit by Product (the money chart)

Visual: Bar chart. **Why:** it exposes loss-makers instantly. **Build:** Axis = `product`; Value = `Total Profit`; sort ascending so losses sit at the bottom. **Conditional formatting:** Format → Bars → Color → *fx* → Rules → if value `< 0` red, else green. **Placement:** bottom-left band.

VISUAL PLACEHOLDER — PROFIT BY PRODUCT

A horizontal bar chart. Most bars are green and point right (profit); one bar (Marketing Suite) is red and points left (loss). A vertical zero line separates them. This is the visual that makes leadership lean in.

Top 10 Customers

Visual: Bar chart. **Build:** Axis = `customer`; Value = `Total Sales`. Add a Top-N filter: Filters pane → `customer` → Filter type "Top N" → Top 10 by `Total Sales`. **Placement:** bottom-right band.

6.3 Page 2 — Product & Discount Deep Dive

Discount Bucket vs Margin (headline insight)

Visual: Clustered column chart. **Why:** it shows the exact discount level where deals stop being profitable. **Build:** Axis = `Discount Bucket` (the column from Chapter 3.6); Value = `Profit Margin %`. Watch the columns fall and go negative past ~20%.

Discount vs Profit scatter

Visual: Scatter chart. **Build:** X = `Avg Discount %`; Y = `Total Profit`; Legend = `segment`; Details = `product`. Reveals which products/segments bleed margin under discount.

Product profitability table

Visual: Table. **Build:** columns `product`, `Total Sales`, `Total Profit`, `Profit Margin %`. **Conditional formatting:** Format → Cell elements → Background color on the margin column → color scale (red → white → green).

Product × Region matrix

Visual: Matrix. **Build:** Rows = `product`, Columns = `region`, Values = `Total Profit`. Spots which product loses money in which region.

6.4 Page 3 — Customer Analysis

Top customers

Bar chart, same pattern as Page 1's top-10 (reuse it here if you like).

Revenue concentration (Pareto)

Visual: Line and clustered column chart. **Build:** Axis = `customer` (Top 15, sorted by sales desc); Column values = `Total Sales`; Line values = `Cumulative % of Sales` (Chapter 5.7). The rising line shows how few customers drive most revenue.

Industry × Segment matrix

Visual: Matrix. Rows = `industry`, Columns = `segment`, Values = `Total Sales` and `Profit Margin %`.

6.5 Slicers & Sync

Add a slicer: Slicer visual → drag a field (e.g., `region`). Set its style to Tile/buttons (Format → Slicer settings → Style) for a clean look; use a Dropdown for long lists. **Sync across pages:** select the slicer → View → **Sync slicers** → tick the pages it should control. Sync Region and Segment across all three pages so filters carry through.

6.6 Drill-Through

1. Add a new page; name it "Customer Detail"; right-click its tab → **Hide page**.
2. On that page, drag `customer` into the Filters pane's **Drill through** well.
3. Build detail visuals (that customer's KPIs, their monthly trend, their orders table).
4. Power BI auto-adds a **Back** button — keep it.
5. Now on any page, right-click a customer in a chart → **Drill through → Customer Detail**.

6.7 Tooltips

Default tooltips: drag extra measures into a visual's Tooltips well to enrich the hover card (e.g., add Profit Margin % to the sales bars). **Report-page tooltips:** build a tiny hidden page, set it as a Tooltip page (Page information → Allow use as tooltip), and assign it to a visual for a rich mini-report on hover.

6.8 Buttons, Bookmarks & Navigation

Page navigation bar

Insert → Buttons → Blank (add three: Overview, Products, Customers). For each: Format button → Action → Type = **Page navigation** → Destination = the matching page. Place the strip at the top of every page.

Reset-filters bookmark

1. Clear all slicers to your default state.
2. View → **Bookmarks** → Add; name it "Reset".
3. Add a button → Action → **Bookmark** → Reset. One click clears filters.

> **SHOW/HIDE WITH BOOKMARKS**
>
> Bookmarks capture the state of visuals (including hidden/shown). Pair two bookmarks with a button to toggle, say, a "Sales view" and a "Profit view" of the same chart — a nice, optional flourish.

7 Design Principles

> *Good design isn't decoration — it's what makes a dashboard readable in seconds. These principles turn a functional report into one that looks like it came from a professional analyst.*

7.1 Color Theory

- **Limit your palette.** One or two brand colors plus neutrals. Too many colors create noise and imply meaning that isn't there.
- **Use color to mean something.** Reserve red/green for bad/good (profit vs loss). Don't color bars rainbow just because you can.
- **Contrast for emphasis.** Make the number that matters the boldest, darkest element; mute everything else.
- **Consistent encoding.** If blue means "sales" on one chart, blue means "sales" everywhere.

7.2 Theme Selection

Apply the project's custom theme once and every visual inherits a consistent palette, fonts, and card style: View → Themes → **Browse for themes** → select `theme.json`. This is faster and more consistent than styling visuals one by one.

7.3 Consistency

Same fonts, same title style, same rounded corners, same spacing on every page. Inconsistency is what makes a dashboard feel amateur even when the analysis is strong.

7.4 Alignment, Spacing & White Space

- **Align to a grid.** Visuals should share edges. Use the alignment tools; nothing ragged or overlapping.
- **Even spacing.** Equal gaps between cards and charts read as intentional.
- **White space is a feature.** Don't fill every pixel. Breathing room guides the eye and reduces overwhelm.

7.5 Accessibility

- **Color-blind safe:** don't rely on red/green alone — pair with position, labels, or icons.
- **Contrast:** dark text on light backgrounds; avoid low-contrast grey-on-grey.
- **Font size:** keep labels legible (avoid tiny 6–7pt text on shared screens).
- **Alt text:** add descriptions to visuals (Format → General → Alt text) for screen readers.

7.6 Professional Design Principles

- **Declutter.** Remove gridlines, heavy borders, and redundant labels. Maximize the "data-ink".
- **Establish hierarchy.** Title > KPIs > charts > detail. Size and weight signal importance.
- **Plain-English titles.** "Profit by Product", not "Sum of profit by product".

- **Add one insight callout.** A small text note ("Discounts above 20% erase margin") shows you interpreted the data, not just charted it.

8 Advanced Features

> *These features move you from "can build a dashboard" to "can run one in production". You won't need all of them for every project, but knowing them is what senior interviewers probe for.*

8.1 Row-Level Security (RLS)

What: restrict which rows a user can see. A regional manager should see only their region. **Build:**

1. Modeling → **Manage roles** → New role, e.g., "AMER Manager".
2. On the `Geography` (or `Sales`) table add a DAX filter: `[region] = "AMER"`.
3. For dynamic RLS, filter by the logged-in user: `[region] = LOOKUPVALUE(UserRegion[region], UserRegion[email], USERPRINCIPALNAME())`.
4. Test with Modeling → **View as** → pick the role.
5. Assign members to the role in the Power BI Service after publishing.

> **WATCH BIDIRECTIONAL FILTERS**
> RLS and bidirectional relationships interact in subtle ways and can leak or over-restrict data. Keep single-direction relationships when using RLS.

8.2 Incremental Refresh

What: refresh only new/changed data instead of reloading everything — essential for large tables. **Build:** create two date parameters named exactly `RangeStart` and `RangeEnd` in Power Query, filter your table between them, then right-click the table → **Incremental refresh** → set the archive window (e.g., store 5 years, refresh the last 10 days). Requires publishing to the Service.

8.3 Performance Analyzer

What: measures how long each visual and its DAX take. **Use:** View → **Performance Analyzer** → Start recording → interact with the page → read the millisecond timings per visual. Long "DAX query" times point to measures to optimize; long "Visual display" times point to overloaded visuals.

8.4 Query Diagnostics

What: profiles what Power Query is doing during refresh (which steps are slow, whether query folding happens). **Use:** in Power Query, Tools → **Diagnose** / Start Diagnostics, refresh, then read the trace to find slow transformation steps.

8.5 Parameters (What-If)

What: a slider users drag to test scenarios. **Build:** Modeling → **New parameter** → What-if → e.g., "Discount Cap" from 0% to 50%. It creates a table and a measure you can reference in DAX to model "what happens to profit if we cap discounts here?".

8.6 Field Parameters

What: let users switch the measure or dimension shown on a single chart from a slicer — one visual does the work of five. **Build:** Modeling → **New parameter** → Fields → add, say, `Total Sales`, `Total Profit`, `Profit Margin %`. Put the resulting field on the chart's value well and add it as a slicer. Users pick the metric; the chart updates.

8.7 Dynamic Formatting

What: change a measure's format string with DAX (e.g., show large numbers as "$1.4M" but small as "$950"). Select a measure → Measure tools → Format → **Dynamic** → write a format-string expression. Keeps big dashboards readable without manual per-visual formatting.

8.8 Mobile Layout

What: a phone-optimized arrangement of the same report. **Build:** View → **Mobile layout** → drag the key visuals (usually the KPI cards and one or two charts) onto the portrait canvas. The desktop layout is untouched; phone users get a tailored view.

8.9 Custom Themes

What: a JSON file defining colors, fonts, and default visual styles. Minimal structure:

```
{
  "name": "SaaS Executive",
  "dataColors": ["#2E5EAA", "#5B9BD5", "#70AD47", "#FFC000", "#C0504D"],
  "background": "#FFFFFF",
  "foreground": "#252525",
  "tableAccent": "#2E5EAA"
}
```

Import via View → Themes → Browse. Editing the JSON is faster and more consistent than restyling visuals individually.

9 Business Insights

> *A dashboard is only worth building if it changes a decision. This chapter teaches you to read each visual, draw conclusions, and turn them into recommendations — the skill interviewers care about most.*

9.1 How to Read Each Chart

Visual	What to look for
KPI cards	The margin card versus the sales card — high sales with low margin means growth isn't translating to profit.
Sales/Profit trend	Direction and gaps. If sales climb but profit is flat, costs or discounts are rising.
By region / segment	Where sales and profit *disagree* — a big-sales, low-profit region is a discounting or cost problem.
Profit by product	Any red bar. A loss-making product is either mispriced, over-discounted, or should be retired.
Discount bucket vs margin	The point where columns cross zero — that's your discount ceiling.
Pareto (concentration)	How steep the cumulative line rises — a steep curve means dangerous dependence on a few customers.
Industry × segment matrix	The cells that are simultaneously high-sales and high-margin — your sweet spot to double down on.

9.2 Business Conclusions (from this dataset)

- **Discounting is the core margin problem.** Margin stays healthy at low discounts and turns negative once discounts pass roughly 20%.
- **At least one product loses money** and drags down the total — it needs pricing action or retirement.
- **Revenue is concentrated** in a small number of top customers, which is a risk if any of them churn.
- **Regions and segments perform unevenly** — the biggest-revenue slice is not always the most profitable.

> **STAY HONEST**
> Quote only numbers your actual data produces. When you plug in the real Kaggle file, the *shape* of these conclusions holds, but the exact figures will differ — use those. Never present illustrative numbers as findings.

9.3 Recommendations

- **Introduce a discount-approval threshold** (e.g., deals above the margin-cliff discount need sign-off).

- **Fix or drop the loss-making product** — reprice, bundle, or retire it.
- **Diversify the customer base** to reduce concentration risk; set a target for revenue share outside the top 10.
- **Reallocate sales effort** toward the high-margin industry/segment cells the matrix reveals.

9.4 Writing the Executive Summary

Lead with the decision, not the method. A good summary is three sentences: *what's happening, why it matters, what to do.* Example:

> **EXAMPLE SUMMARY**
> "The business is growing revenue but margin is under pressure: deals discounted above 20% are unprofitable and one product line runs at a loss. Revenue also depends heavily on a handful of customers. We recommend a discount-approval threshold, a pricing review of the loss-making product, and a push to broaden the customer base."

9.5 The Decision-Making Process

1. **Observe** a pattern on the dashboard (margin falls with discount).
2. **Ask why** and form a hypothesis (reps discount to close, eroding profit).
3. **Act** with a specific change (approval threshold above X% discount).
4. **Measure** the effect next quarter on the same dashboard (did margin recover?).

> **INTERVIEW ANGLE**
> Practice narrating one insight end to end: the visual, the number, the business meaning, and the action you'd recommend. That four-part story is what turns "I made charts" into "I drive decisions".

10 Common Mistakes

> *A consolidated list of the traps that catch beginners and cost marks in interviews. If you internalize this chapter, you'll avoid most of the pain.*

10.1 Beginner Mistakes

- Building visuals before the data model exists (charts are only as right as the model beneath them).
- Using one giant flat table instead of a star schema.
- Forgetting to mark the Date table, then wondering why YoY/YTD misbehave.
- Cleaning data manually in the report instead of upstream (no repeatability).

10.2 Performance Issues

Cause	Fix
Too many visuals on one page	Split across pages; use drill-through for detail
Bidirectional relationships everywhere	Default to single-direction
Heavy calculated columns	Prefer measures; push static derivations to Power Query/ source
High-cardinality columns in the model	Remove unused ID/text columns (e.g., license)
Row-by-row DAX on big tables	Use set-based functions; check with Performance Analyzer

10.3 Incorrect DAX

- `COUNT` instead of `DISTINCTCOUNT` for orders/customers — overstates the count.
- Averaging a row-level ratio column instead of computing `DIVIDE(SUM, SUM)` — wrong margin.
- Using `/` instead of `DIVIDE` — divide-by-zero errors.
- Not understanding filter context — measures return "unexpected" numbers that are actually correct for the context.

10.4 Relationship Problems

- Ambiguous paths from too many bidirectional relationships.
- Inactive relationships forgotten (needing `USERELATIONSHIP` to activate in a measure).
- Joining on a text name instead of a stable ID.
- Wrong cross-filter direction, so slicers don't filter what you expect.

10.5 Visualization Mistakes

- Pie charts with many slices (hard to compare — use a bar chart).
- 3-D charts and gratuitous effects (they distort perception).
- Truncated or inconsistent axes that exaggerate differences.

- Rainbow colors with no meaning; dual axes that mislead.
- Cryptic default titles ("Sum of profit by product").

10.6 Best-Practice Recap

> **THE SHORT LIST**
>
> Star schema · marked Date table · single-direction filters · measures over columns · `DIVIDE` and `DISTINCTCOUNT` · one story per page · consistent theme · plain-English titles · one insight callout per page · honest numbers.

11 Practice Exercises & Interview Prep

> *Reading is not learning — building is. Work these exercises in your own file, then rehearse the interview answers out loud.*

11.1 Practice Questions

1. In Power Query, create a `Discount Bucket` column using a Conditional Column (no code). Verify it matches the M version in Chapter 3.6.
2. Write a measure for **Profit per Customer** = Total Profit ÷ Total Customers.
3. Write a **Sales QoQ %** (quarter-over-quarter) measure using `DATEADD(..., -1, QUARTER)`.
4. Build a measure that returns **Total Sales for AMER only**, regardless of the region slicer, using `CALCULATE` + `ALL`.
5. Create a **dynamic subtitle** that shows the selected segment or "All Segments".

11.2 Dashboard Challenges

1. Add a fourth page: "Regional Deep Dive" with a map visual (Country → Sales) and a region slicer.
2. Add a **field parameter** so users can switch the profit-by-product chart between Sales, Profit, and Margin %.
3. Add a **report-page tooltip** to the region bar showing that region's top 3 products on hover.
4. Build a **What-If discount cap** slider and a measure estimating profit under the cap.

11.3 Mini Assignments

- Recreate Page 1 from a blank file in under 30 minutes using only your own notes.
- Apply the custom theme, then redesign one page to improve alignment and white space.
- Add RLS so a test "AMER Manager" role sees only AMER rows; verify with "View as".

11.4 Interview Questions

Power BI & modeling

- What is a star schema and why is it preferred over a flat table?
- Difference between a measure and a calculated column? When do you use each?
- Why mark a Date table, and what breaks if you don't?
- Single vs bidirectional cross-filter — when would you use bidirectional?

DAX

- How do you count orders when one order has many rows? (DISTINCTCOUNT)
- Why compute margin as `DIVIDE(SUM,SUM)` and not the average of a margin column?
- Explain filter context with an example from this dashboard.
- How does `SAMEPERIODLASTYEAR` work, and what does it depend on?

Business

- Walk me through the discount-vs-margin finding and what you'd recommend.
- How would you explain revenue concentration risk to a non-technical VP?

- What would you build next if given another two weeks?

11.5 Real Business Scenarios

- "The CFO wants discount governance." What visuals and threshold do you present, and how do you justify the cutoff?
- "Sales are up but the board is worried about profit." Which two visuals tell that story fastest?
- "Our biggest customer is renegotiating." How does your dashboard quantify the risk?
- "Which market should we invest in next quarter?" How do you use region and margin together to answer?

A Appendix — Cheat Sheets & Glossary

> *Quick reference for when you're building and can't remember the exact syntax or term.*

A.1 Measure Cheat Sheet

Measure	Formula (short)
Total Sales	SUM(Sales[sales])
Total Profit	SUM(Sales[profit])
Total Orders	DISTINCTCOUNT(Sales[order_id])
Total Customers	DISTINCTCOUNT(Sales[customer_id])
Profit Margin %	DIVIDE([Total Profit],[Total Sales])
Avg Order Value	DIVIDE([Total Sales],[Total Orders])
Avg Discount %	AVERAGE(Sales[discount])
Sales LY	CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
Sales YoY %	DIVIDE([Total Sales]-[Sales LY],[Sales LY])
Sales YTD	TOTALYTD([Total Sales],'Date'[Date])
Loss-Making Sales	CALCULATE([Total Sales], Sales[profit] < 0)
Product Rank	RANKX(ALL(Sales[product]),[Total Sales],,DESC)

A.2 Useful Keyboard Shortcuts

Action	Shortcut
Copy / paste a visual	Ctrl+C / Ctrl+V
Multi-select visuals	Ctrl+click
Nudge a visual	Arrow keys
Open Selection pane	View → Selection
Save	Ctrl+S

A.3 Glossary

Term	Meaning
Measure	A DAX calculation evaluated live for the current filter context.
Calculated column	A DAX column computed row-by-row and stored in the model.
Filter context	The set of filters (slicers, axis, row) applied when a measure is evaluated.
Fact table	The table of measured events (our Sales).
Dimension table	Descriptive context you slice by (Date, Customer, Product, Geography).
Degenerate dimension	An ID kept in the fact table with no dimension of its own (order_id).
Star schema	One fact table joined directly to surrounding dimensions.
Cardinality	How rows relate across a relationship (usually many-to-one).
Cross-filter direction	Which way filters travel across a relationship (single or both).
Power Query (M)	The tool/language that loads and transforms data at refresh.
DAX	The language for measures and calculated columns.
RLS	Row-Level Security — restricts which rows a user can see.

YOU'RE READY

Work through this guide once end to end while building, and you'll have a portfolio-quality dashboard you fully understand — no external tutorials needed. When you plug in the real Kaggle data, refresh your numbers and update the insight callouts to match. Then rehearse the Chapter 11 interview answers out loud.